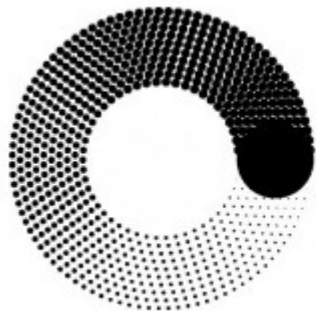




**МОСКОВСКИЙ
ПОЛИТЕХ**

**АВТОМАТИЗАЦИЯ ПРОЦЕССОВ ЖИЗНЕННОГО
ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ**

**Лабораторная работа № 6.
Статический анализ кода**

**Москва
2024**

Теоретические основы

Статический анализ кода (Static Code Analysis) — анализ исходного кода программного обеспечения, производимый без реального выполнения исследуемых программ (в отличие от динамического анализа). В большинстве случаев анализ производится над исходным кодом, хотя, иногда анализу подвергается объектный код, например Р-код или код на MSIL. Термин обычно применяют к анализу, производимому специальным программным обеспечением (ПО), тогда как ручной анализ называют «program understanding», «program comprehension» (пониманием или постижением программы).

В зависимости от используемого инструмента глубина анализа может варьироваться от определения поведения отдельных операторов до анализа всего имеющегося кода. Способы использования полученной в ходе анализа информации также различны — от выявления мест, возможно содержащих ошибки (утилиты типа Lint), до формальных методов, позволяющих математически доказать какие-либо свойства программы (например, соответствие поведения спецификации).

Типы ошибок, обнаруживаемых статическими анализаторами

- **Неопределённое поведение** — неинициализированные переменные, обращение к NULL-указателям. О простейших случаях сигнализируют и компиляторы.
- **Нарушение алгоритма пользования библиотекой.** Например, во многих языках программирования каждому открытию файла должно соответствовать его закрытие, поэтому если файловая переменная теряется раньше, чем файл закрывается, анализатор может сообщить об ошибке.
- **Сценарии, приводящие к недокументированному поведению.**
- **Переполнение буфера,** чаще всего вызванное использованием небезопасных функций или неопределённым поведением.
- **Типичные сценарии, мешающие кроссплатформенности.**

Задания

1. Провести исследования доступных статических анализаторов для выбранного командой стека разработки (большой, но не всеобъемлющий список ([https://owasp.org/www-community/Source Code Analysis Tools](https://owasp.org/www-community/Source_Code_Analysis_Tools)))
2. Выбрать минимум два из следующих видов статических анализаторов:
 - а. анализатор качества кода, например, Checkstyle
 - б. анализатор безопасности кода, например, Gosec
 - с. средство поиска учетных данных/секретов, например, Trufflehog.
3. Добавить в построенном пайплайне TeamCity шаги, на которых код проверяется статическими анализаторами и, в случае обнаружения проблем максимальной критичности для выбранного анализатора (для анализатора безопасности — CRITICAL, для анализатора качества кода — BLOCKER, для средства поиска секретов — любой обнаруженный пароль), билд завершается с ошибкой.
4. Подготовить отчет по лабораторной работе (текстовый документ и скриншоты с описанием по вышеописанным пунктам).